

LECTURE 9

FUNDAMENTAL DESIGN

PRINCIPLES AND

ENTERPRISE SYSTEMS

ARCHITECTURE DESIGN

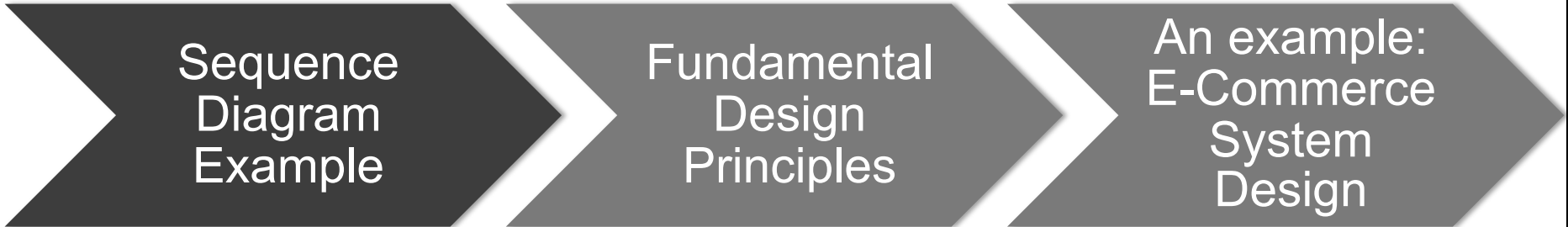
LEK HSIANG HUI

LEARNING OBJECTIVES

At the end of this lecture, you should understand:

- The fundamental design principles
- How to design enterprise systems

SEQUENCE DIAGRAM EXAMPLE



Sequence
Diagram
Example

Fundamental
Design
Principles

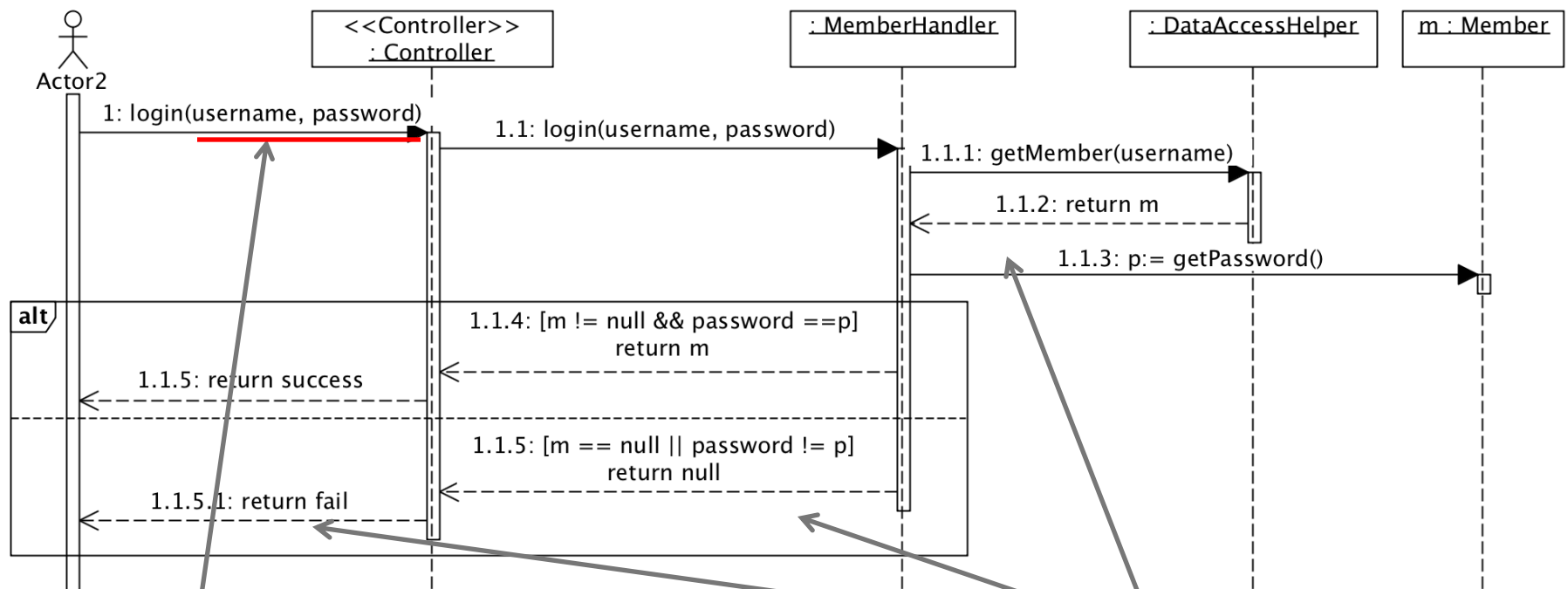
An example:
E-Commerce
System
Design

TASK : LOGIN

Come up with the sequence diagram for a typical login use case

- Idea:
- Actor interact with a Controller class
- The Controller then call the login() method in MemberHandler class which contains a list of member-related operations
- In order to access the database, we use a DataAccessHelper class
 - Retrieve the user by the username
 - Compare the password supplied by the user to that found in the database

TASK : LOGIN



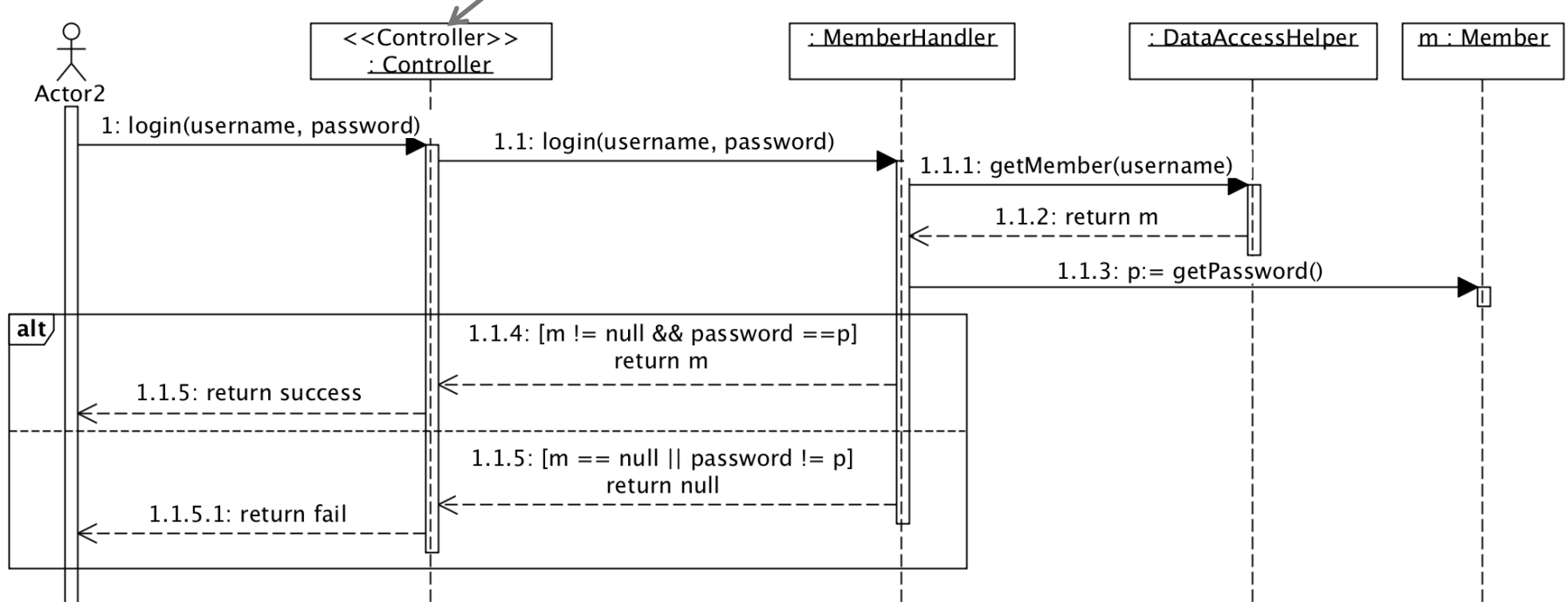
User need to supply the input data (username, password)

Return values must only be sent back to the caller

TASK : LOGIN

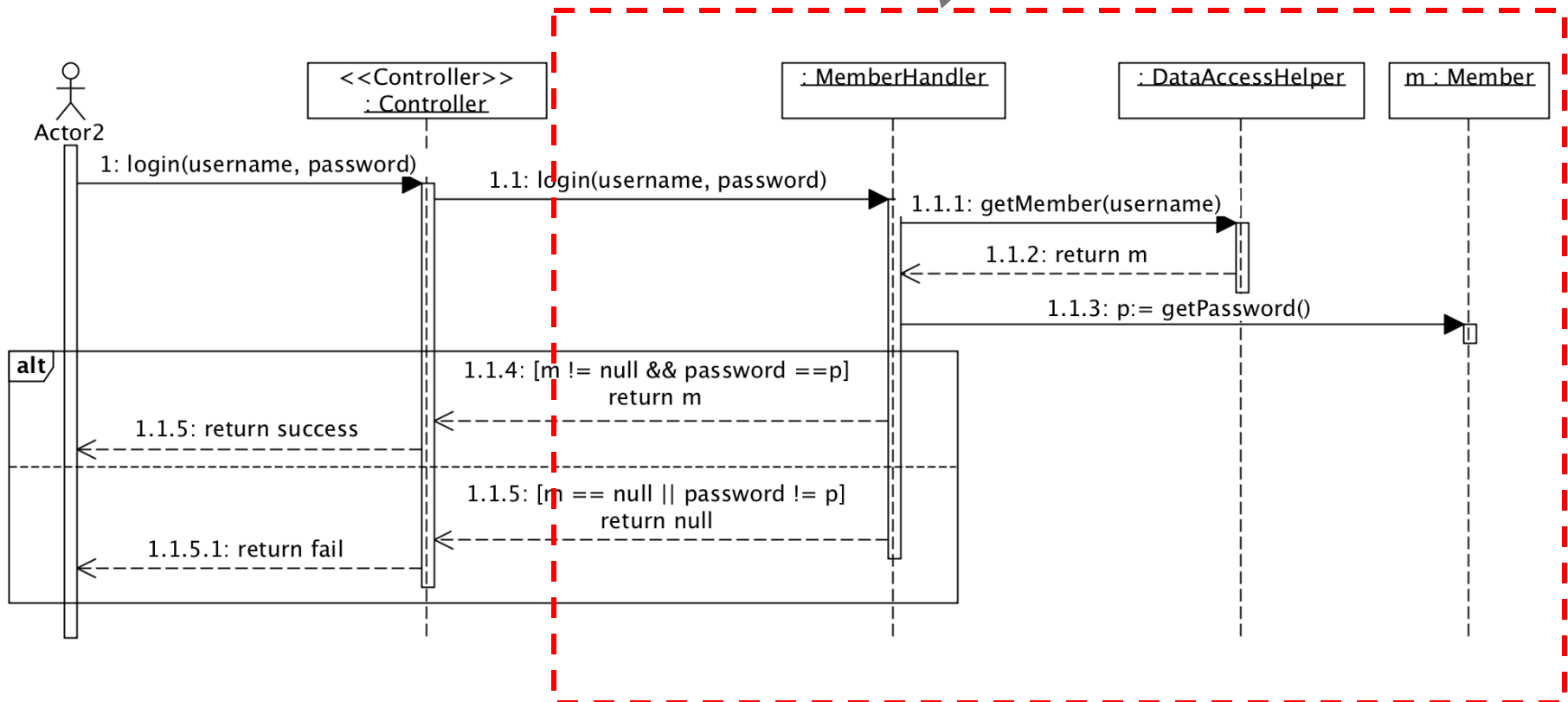
Controller is the middle man to:

- Direct requests to the relevant domain model classes (e.g. "Handlers") to get the result
- Forward user to the view with the result

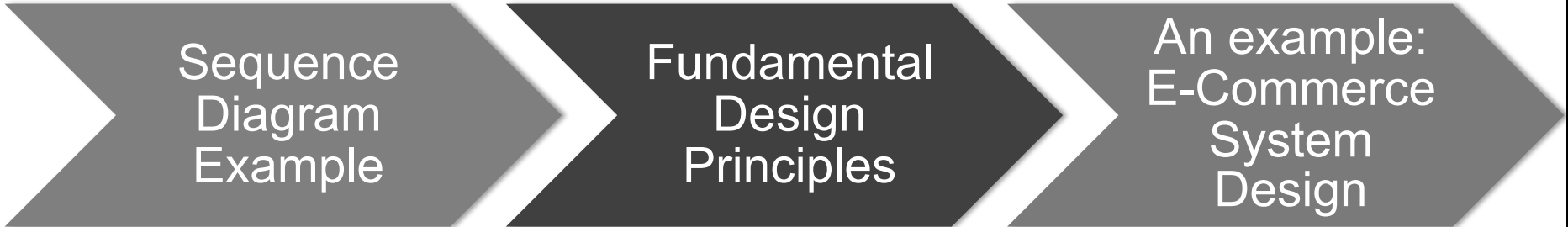


TASK : LOGIN

This part is the most important, sometimes for convenience, we might also omit the controller class



FUNDAMENTAL DESIGN PRINCIPLES

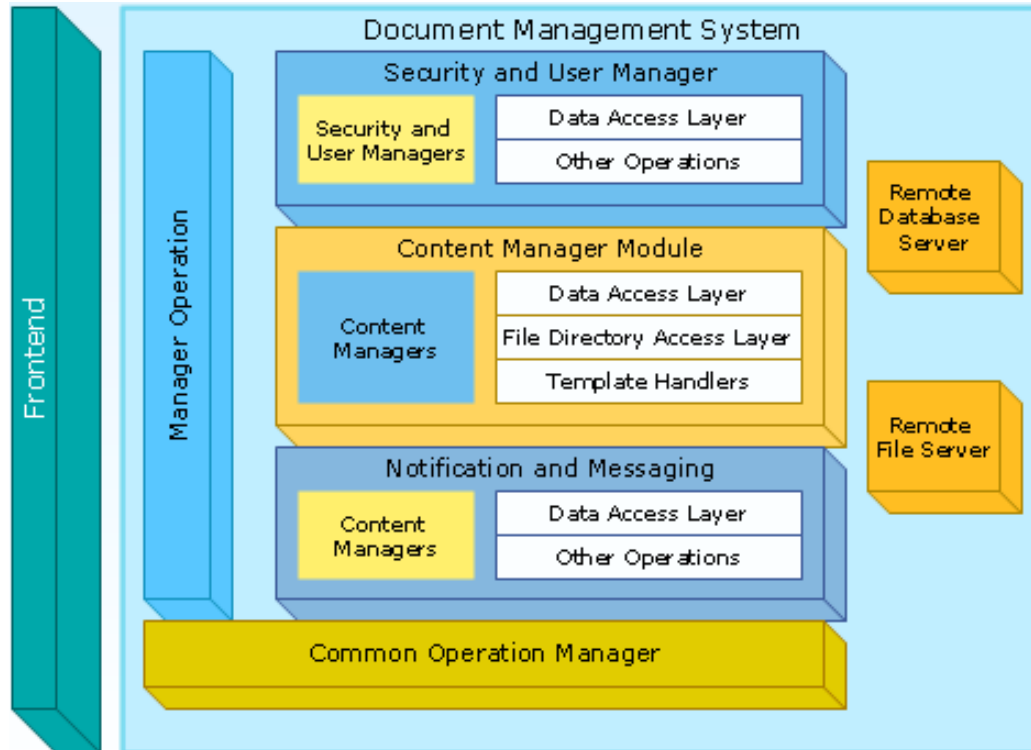


Sequence
Diagram
Example

Fundamental
Design
Principles

An example:
E-Commerce
System
Design

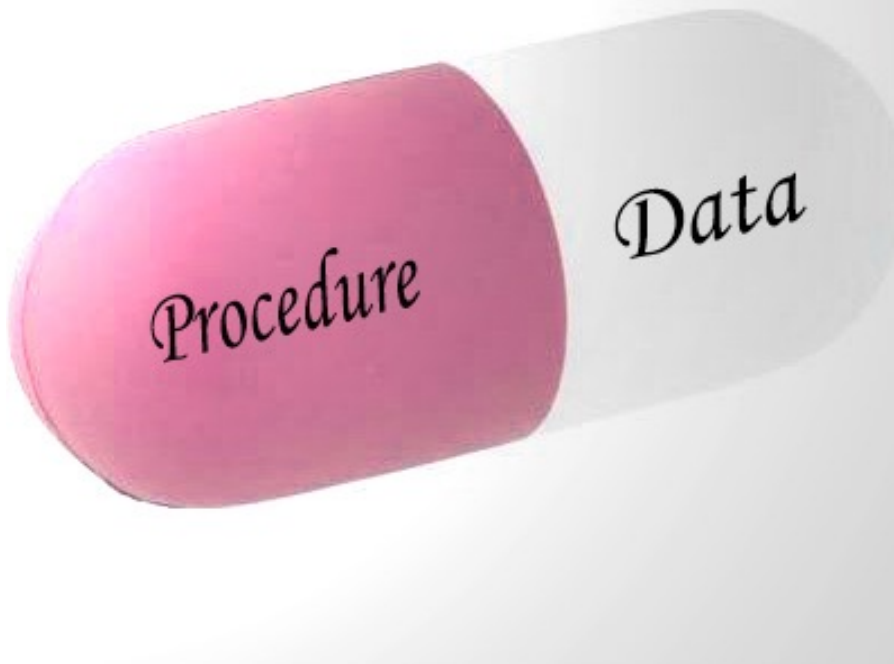
MODULARITY



Breaking the system into small modules

ENCAPSULATION

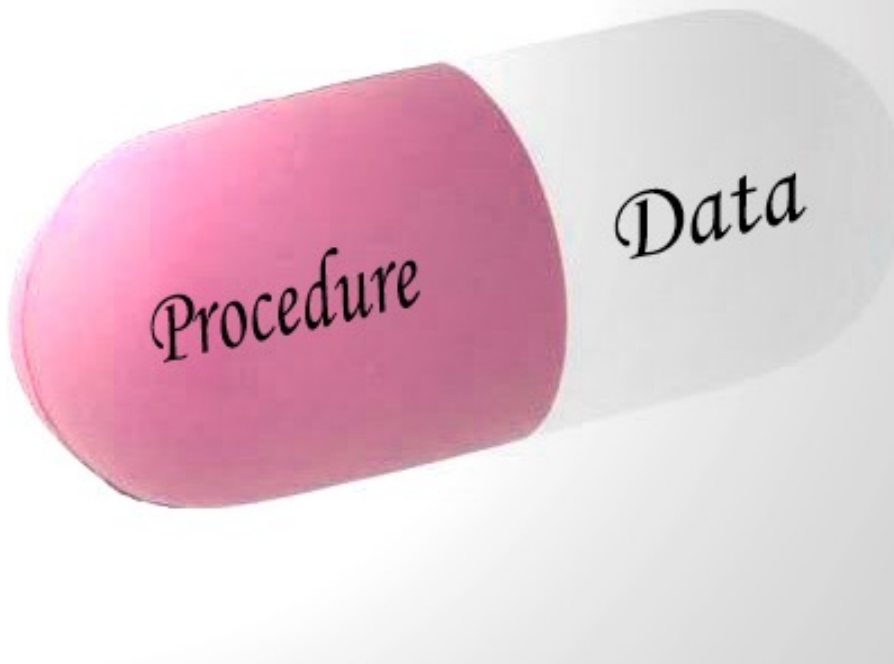
Encapsulation



Design concept that each object is a self-contained unit containing both data and program logic

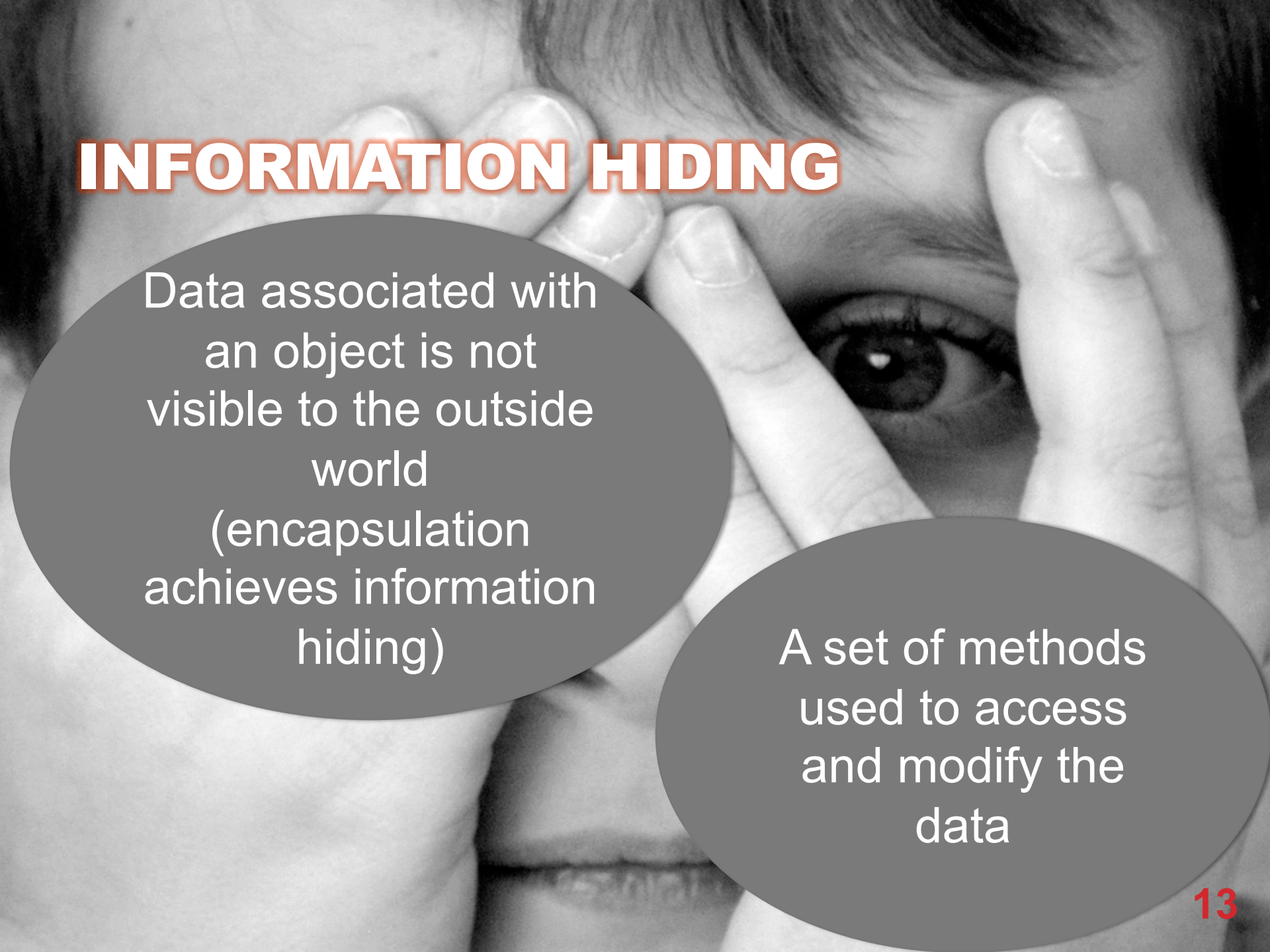
ENCAPSULATION

Encapsulation



Each object provides a set of services that are invoked by calling the object's methods

INFORMATION HIDING



Data associated with
an object is not
visible to the outside
world
(encapsulation
achieves information
hiding)

A set of methods
used to access
and modify the
data

NAVIGATION VISIBILITY

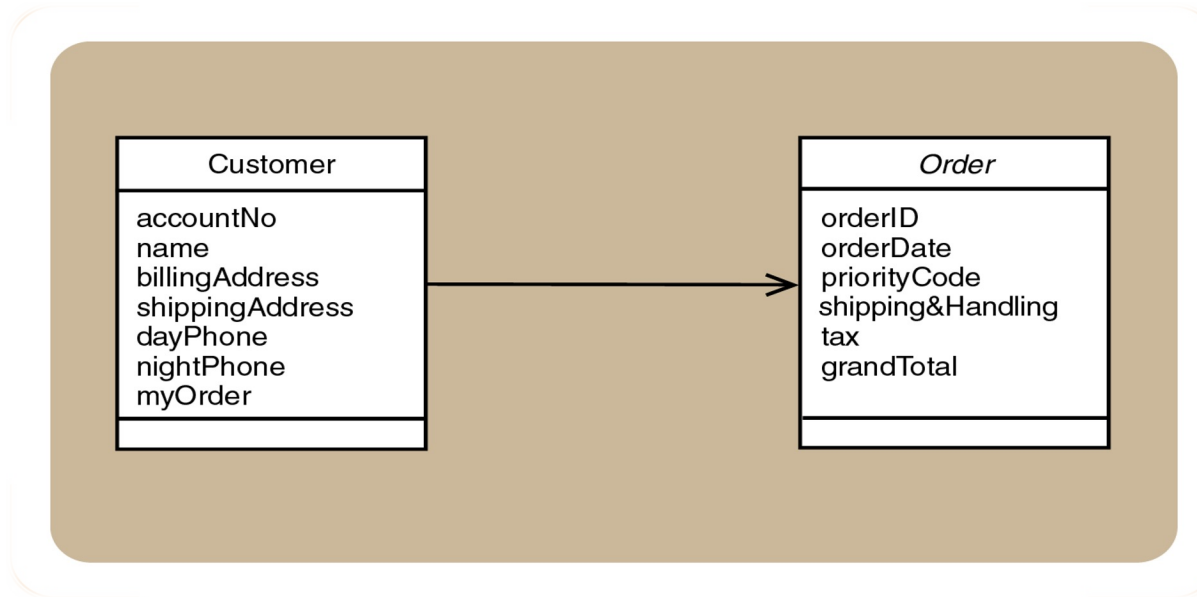
A man with short dark hair, wearing a light green ribbed sweater, is looking through a pair of black binoculars. He is standing on a balcony with a metal railing. In the foreground, there are green leafy plants. In the background, there is a residential area with white buildings and red-tiled roofs, and a hillside with more buildings in the distance. A white and green striped cloth is hanging on the right side of the balcony.

Refers to the ability
of an object to
view/interact with
another object

NAVIGATION VISIBILITY

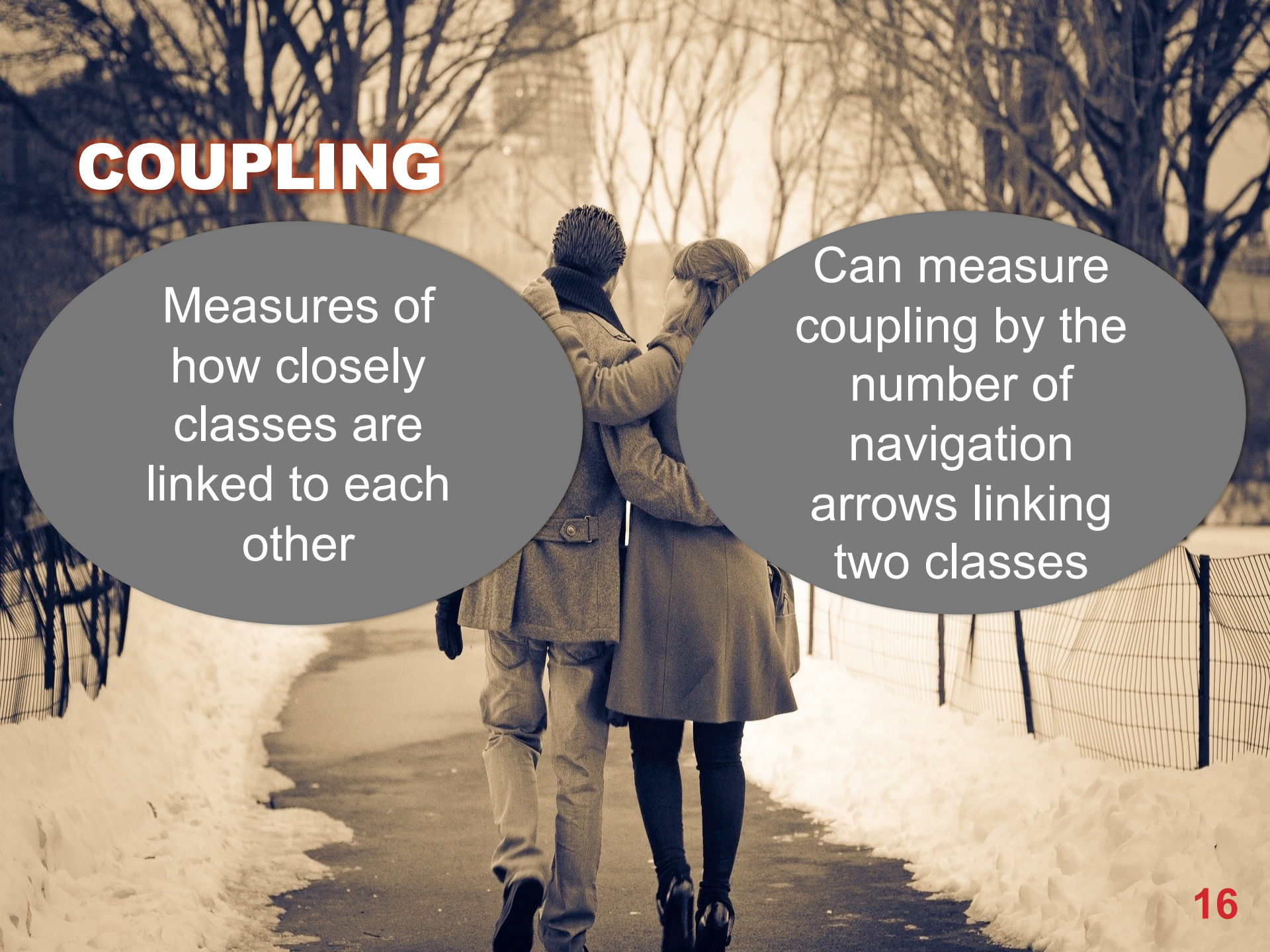
Interactions between objects can only be accomplished with navigation visibility

One of the responsibilities of a design is to specify which classes have navigation visibility to other classes



One way navigation
visibility between
Customer class and
Order class

COUPLING

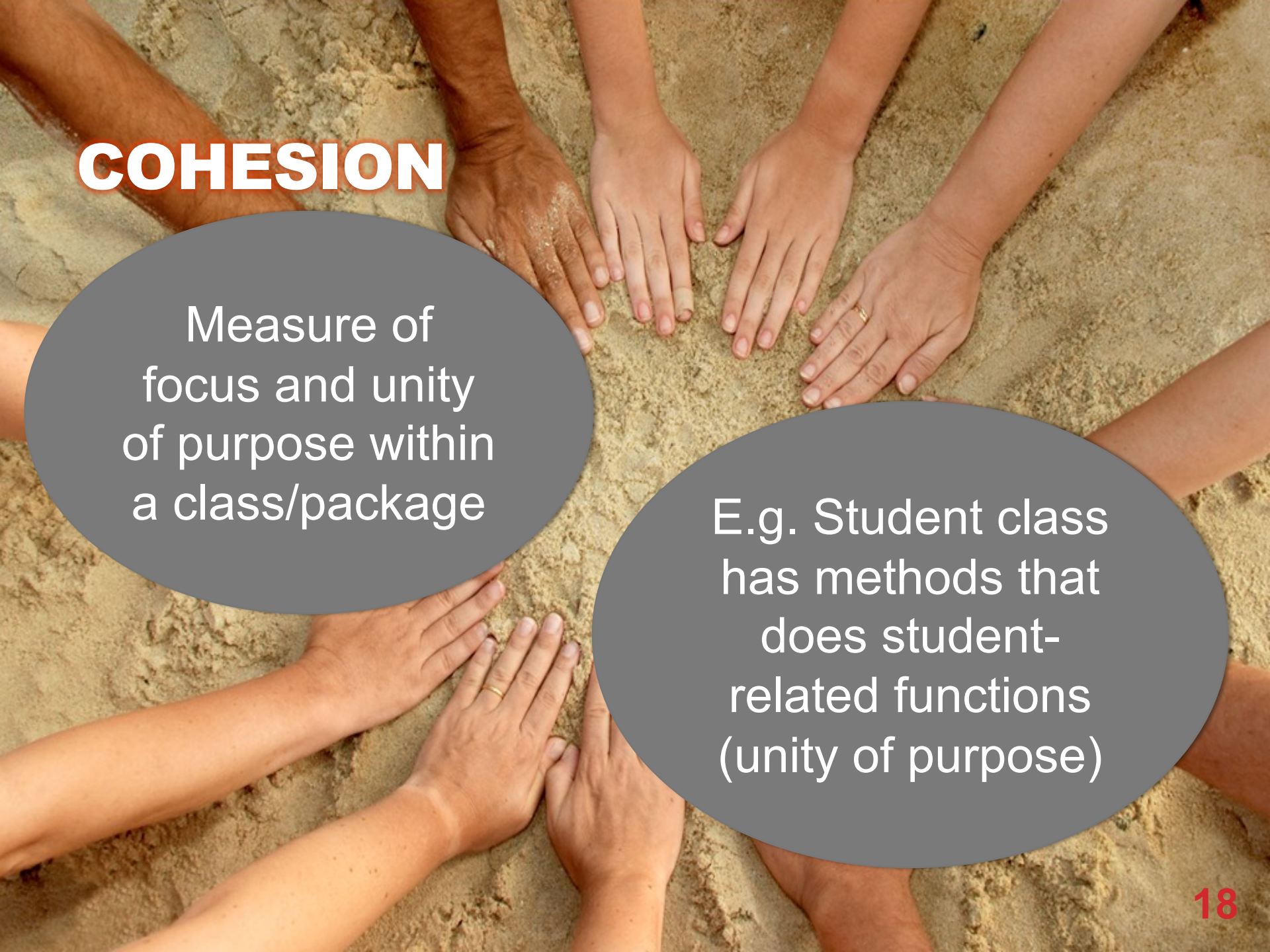
A romantic couple walking away from the camera on a snow-covered path. The man is wearing a dark coat and the woman is wearing a light-colored coat. They are surrounded by bare trees and a fence, suggesting a park or garden in winter.

Measures of how closely classes are linked to each other

Can measure coupling by the number of navigation arrows linking two classes

Is tight or loose coupling better?

COHESION



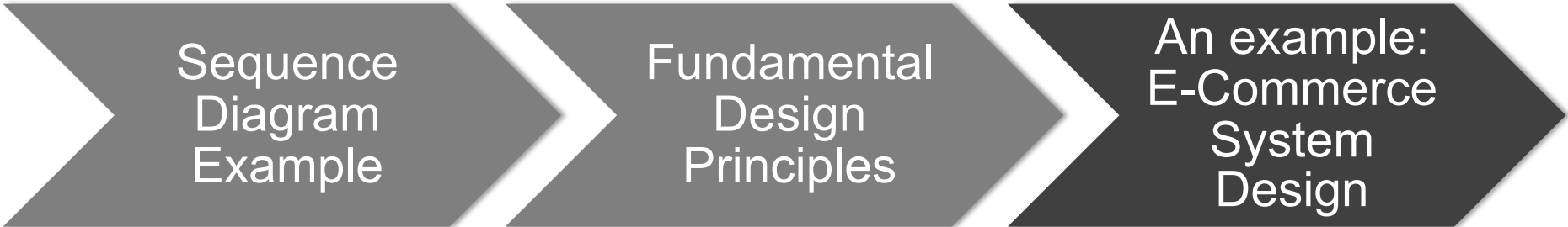
Measure of
focus and unity
of purpose within
a class/package

E.g. Student class
has methods that
does student-
related functions
(unity of purpose)

How is cohesion different from coupling?

Is low or high cohesion better?

AN EXAMPLE: E-COMMERCE SYSTEM DESIGN



Sequence
Diagram
Example

Fundamental
Design
Principles

An example:
E-Commerce
System
Design

CRUD

Most system should at least contain the CRUD operations:

- **Create** – e.g. Create a Listing
- **Read** – e.g. Search for Listings
- **Update** – e.g. Update a Listing
- **Delete** - e.g. Delete Listing

USE CASE: CREATE ONLINE ORDER

When crafting the use case description/sequence diagram, should think in a logical manner from the **user's perspective**

- Users will not be able to know item ids
- They have to search by a keyword and the system will return a list of item ids

(Sequence Diagram) System = backend-related operations only

- Frontend operations which do not connect with the backend is ignored
- E.g. often when adding items to the “shopping cart”, it is only updated in the browser → nothing is sent to the server
- So these operations are not documented

USE CASE: CREATE ONLINE ORDER

We often refer the System section to just backend-related operations

- ...
- Once we have the list of items, user will click on checkout cart which will send the list of all the selected items
- Each item is identified using listingId (together with the relevant information such as quantity, color, etc)
- Some operations such as credit card verification might be **asynchronous** (i.e. the system will still go on to perform but if things go wrong, it will be an exception condition since the use case cannot be fulfilled)

Use case name	Create online order	
...	...	
Actors	Customers	
Pre-conditions	Customer is logged in	
...	...	
Flow of events	Actor	System
	1. Searches for item using keywords or browse item categories 2. Adds item to shopping cart with quantity and color 3. Repeat step 1 and 2 until all items are added 4. Checks out cart (submit list of item ids, quantities) 5. Enters credit card details	1.1 Displays item listings 5.1 Retrieve the list of listings 5.2 Verifies enough quantity 5.3 Create order transaction 5.4 Add payment verification queue

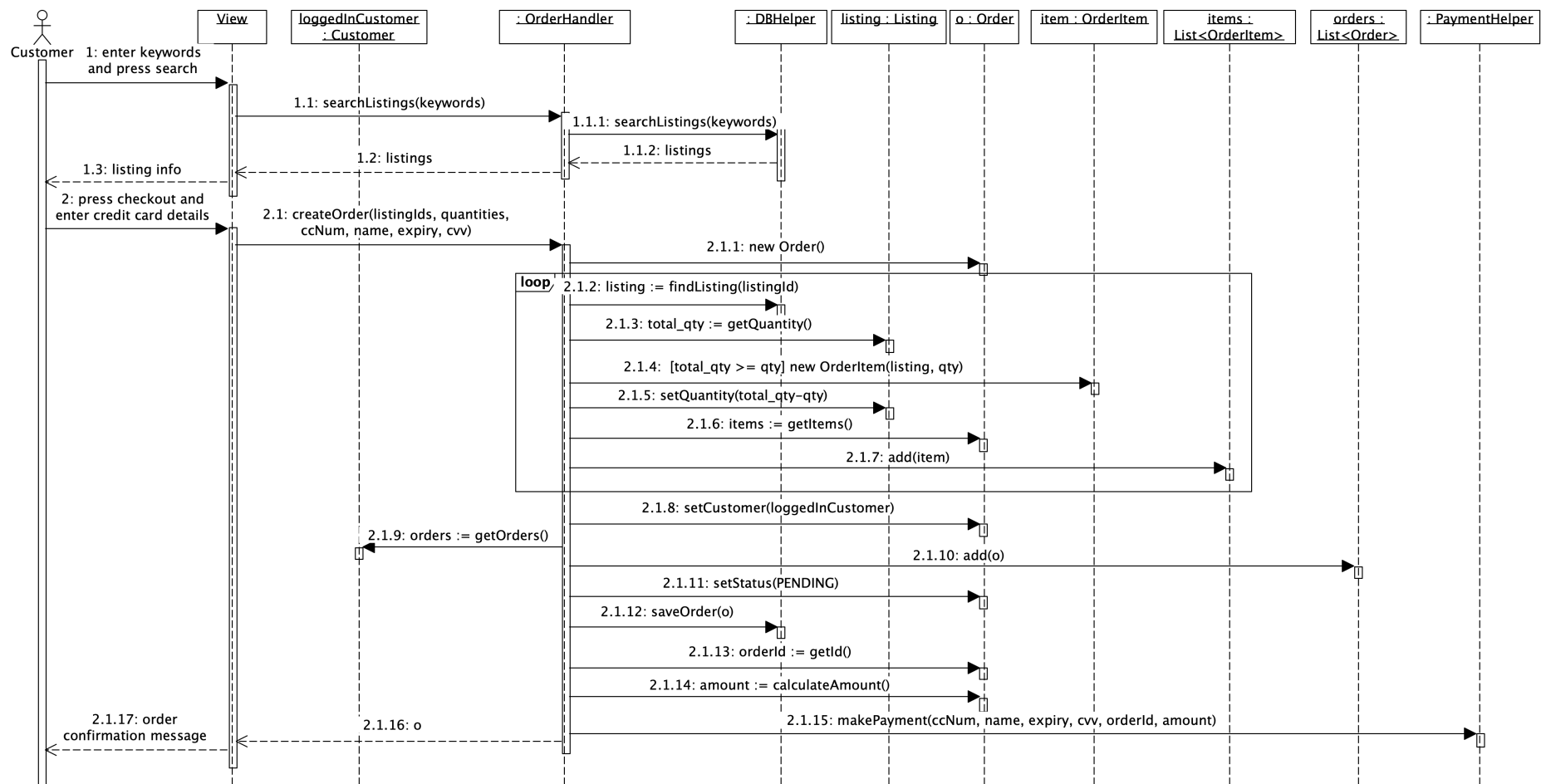
Use case name	Create online order	
...	...	
Actors	Customers	
Pre-conditions	Customer is logged in	
...	...	
Flow of events	Actor	System
	1. Searches for item using keywords or browse item categories 2. Adds item to shopping cart with quantity and color 3. Repeat step 1 and 2 until all items are added 4. Checks out cart (submit list of item ids, quantities) 5. Enters credit card details	1.1 Displays item listings Include all the logic needed to get the “input data” for the use case 5.1 Retrieve the list of listings 5.2 Verifies enough quantity 5.3 Create order transaction 5.4 Add payment verification queue

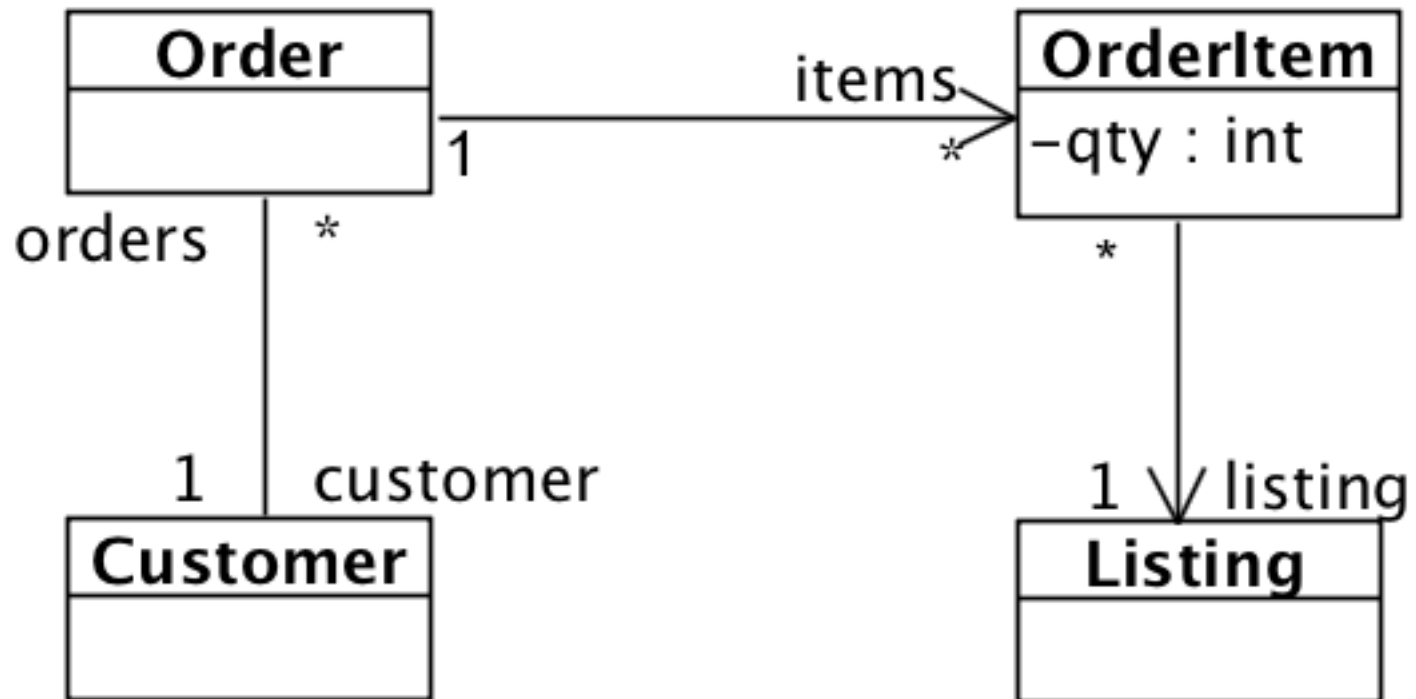
Use case name	Create online order	
...	...	
Actors	Customers	
Pre-conditions	Customer is logged in	
...	...	
Flow of events	Actor	System
	1. Searches for item using keywords or browse item categories 2. Adds item to shopping cart with quantity and color 3. Repeat step 1 and 2 until all items are added 4. Checks out cart (submit list of item ids, quantities) 5. Enters credit card details	1.1 Displays item listings 5.1 Retrieve the list of listings 5.2 Verifies enough quantity 5.3 Create order transaction 5.4 Add payment verification queue

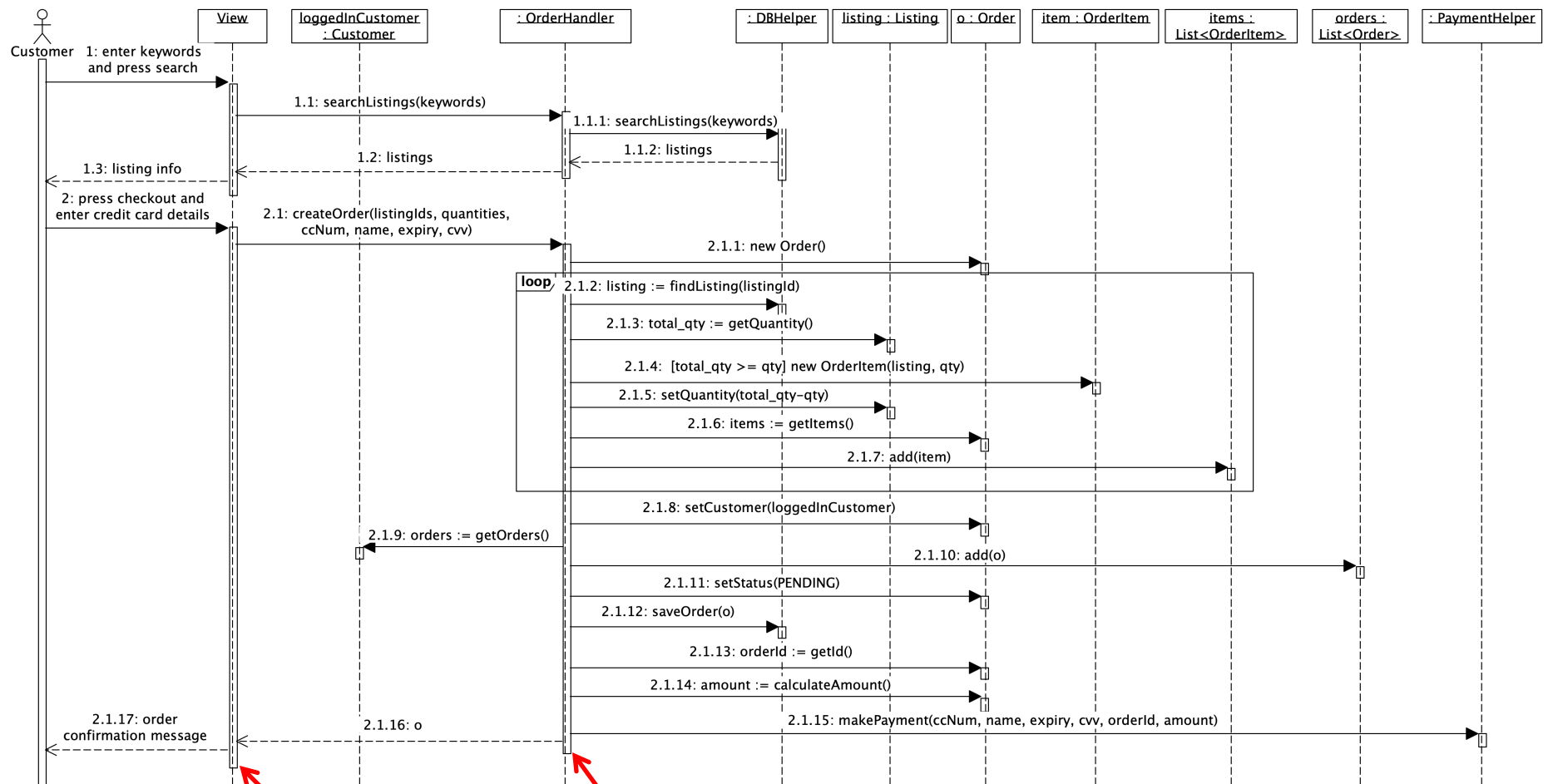
The flow of event should not just end at create transaction but should also include any operations that needs to happen (even though it is unseen by the user)

Use case name	Create online order	
...	...	
Actors	Customers	
Pre-conditions	Customer is logged in	
...	...	
Flow of events	Actor	System
	1. Searches for item using keywords or browse item categories 2. Adds item to shopping cart with quantity and color 3. Repeat step 1 and 2 until all items are added 4. Checks out cart (submit list of item ids, quantities) 5. Enters credit card details	1.1 Displays item listings Credit card verification is asynchronous process: • We send a request to the bank which will then reply at a later stage • Order is currently <u>PENDING</u> 5.1 Retrieve the list of listings 5.2 Verifies enough quantity 5.3 Create order transaction 5.4 Add payment verification queue

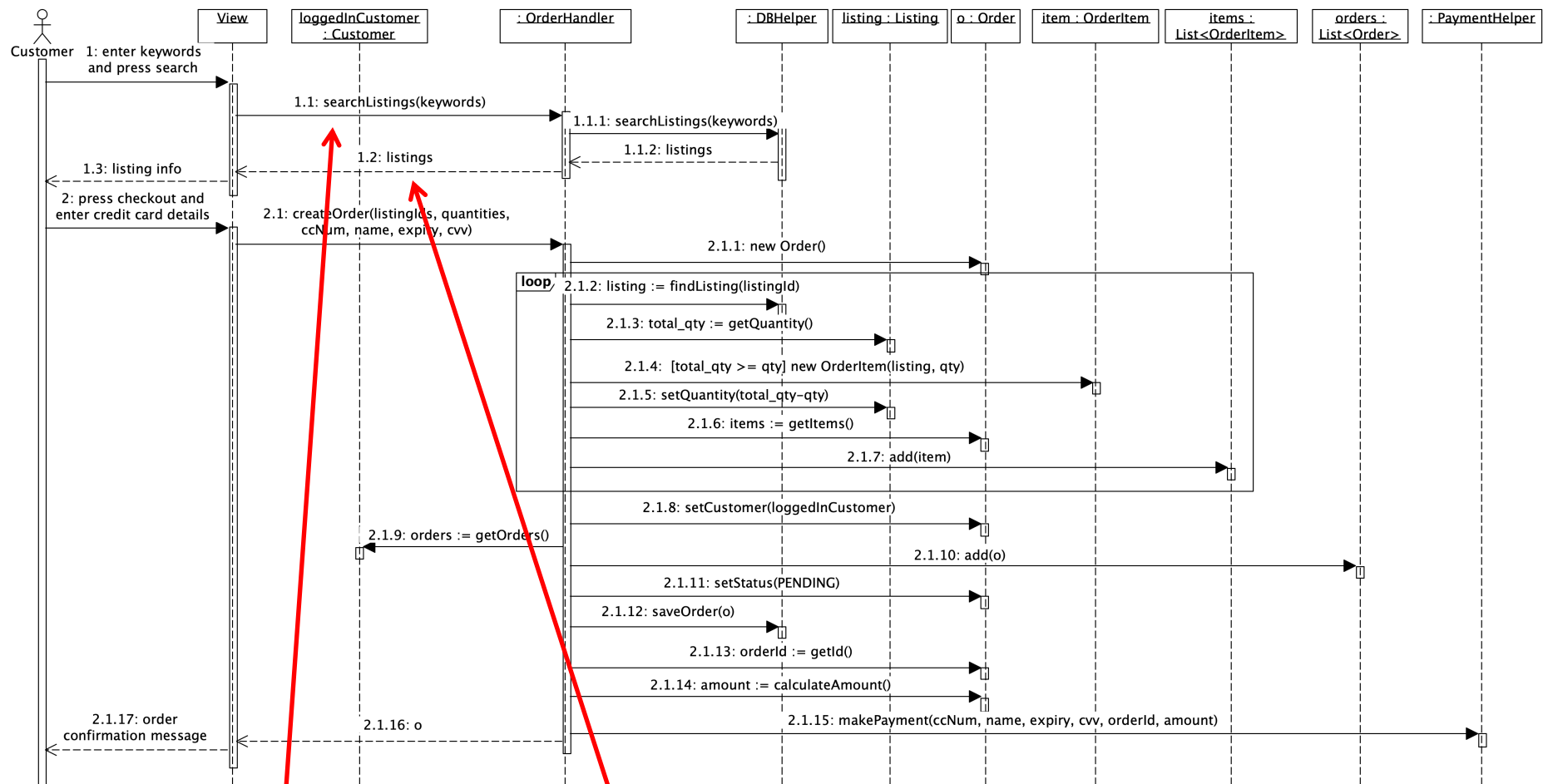
Use case name	Create online order	
...	...	
Actors	Customers	
Pre-conditions	Customer is logged in	
...	...	
Flow of events	Actor	System
	1. Searches for item using keywords or browse item categories 2. Adds item to shopping cart with quantity and color 3. Repeat step 1 and 2 until all items are added 4. Checks out cart (submit list of item ids, quantities) 5. Enters credit card details	1.1 Displays item listings Credit card verification is asynchronous process: Receiving the payment verification confirmation is another use case which will update the order status to be <u>PAID</u> 5.1 Retrieve the list of listings 5.2 Verifies enough quantity 5.3 Create order transaction 5.4 Add payment verification queue



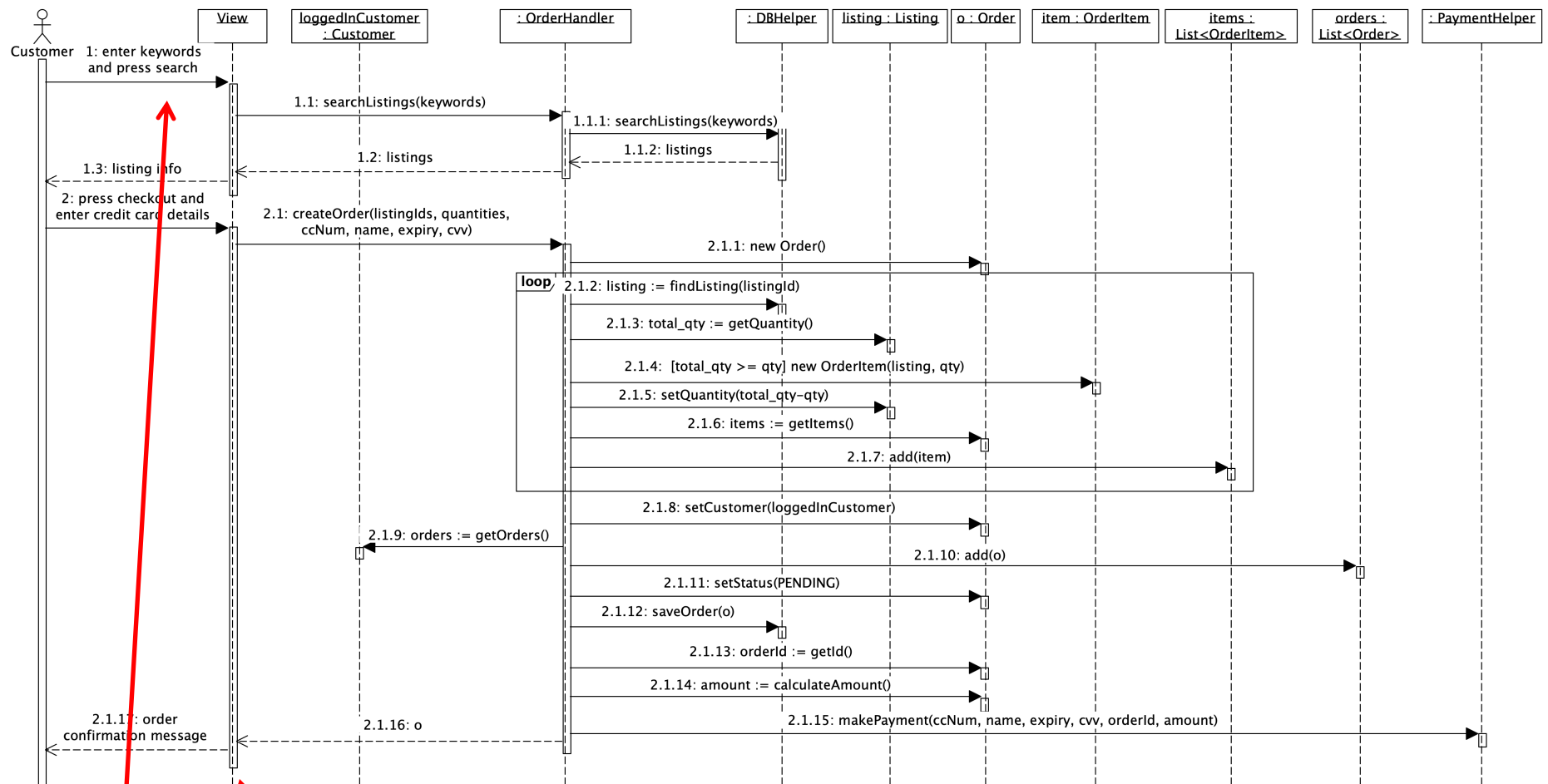




Note that the left bar should exist before the right bar and should be longer than the right bar (caller should exist longer than the callee)

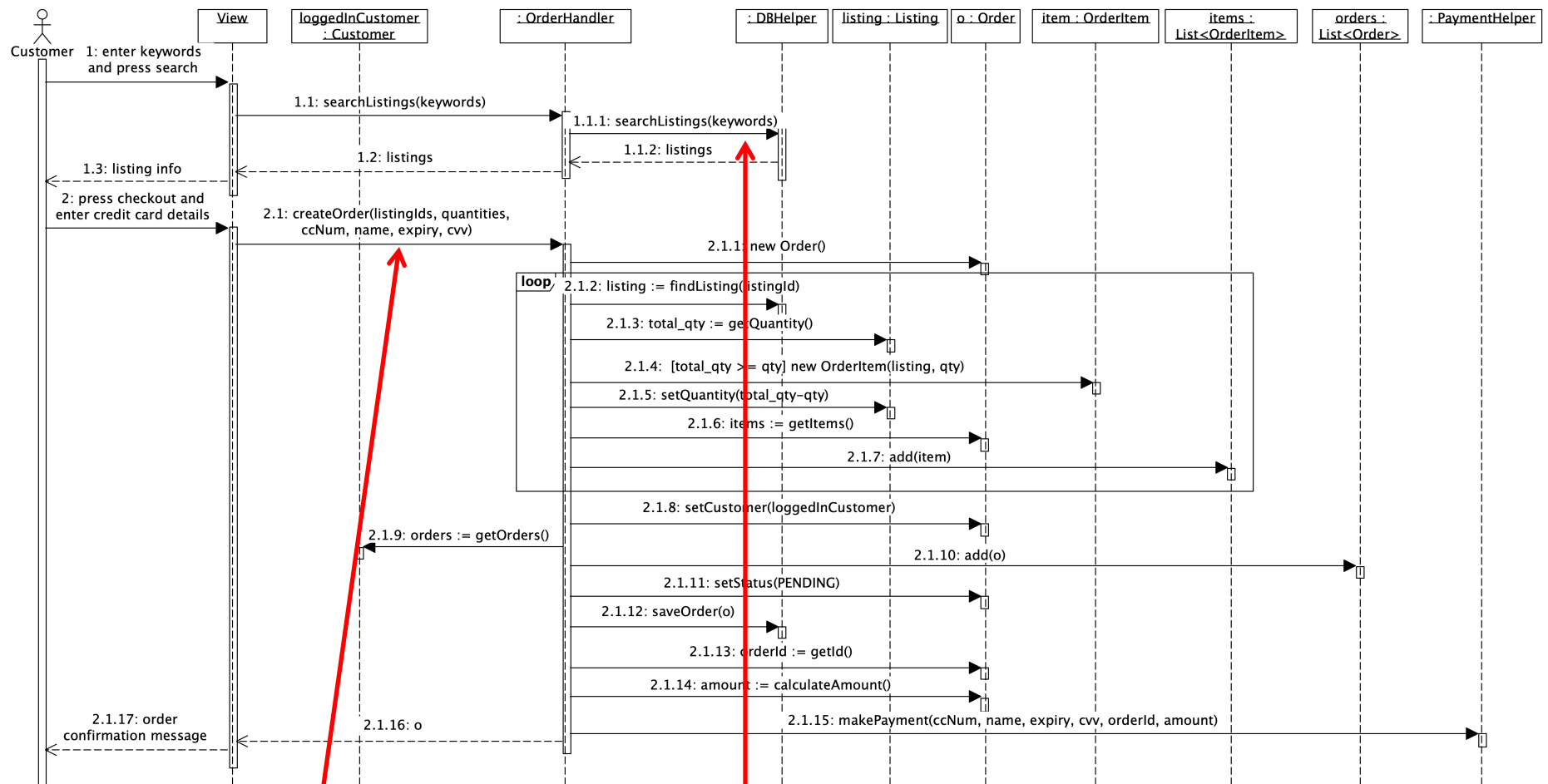


The return message (if any) must be always sent back to the caller
 Or use the alternative notation:
 listings := searchListings(keywords)

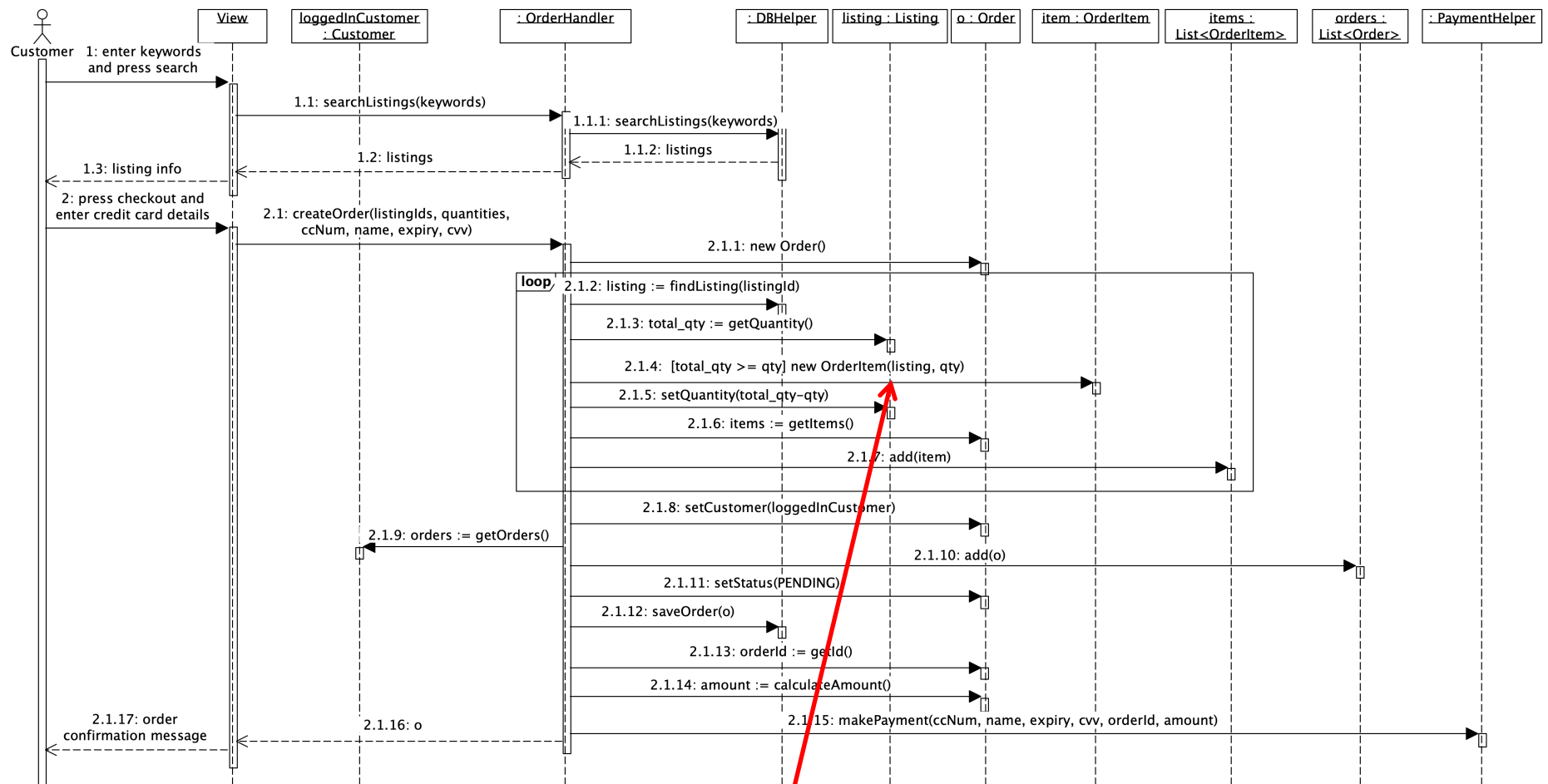


We often omit the view lifeline since it does not involve any method calls but rather what the user does

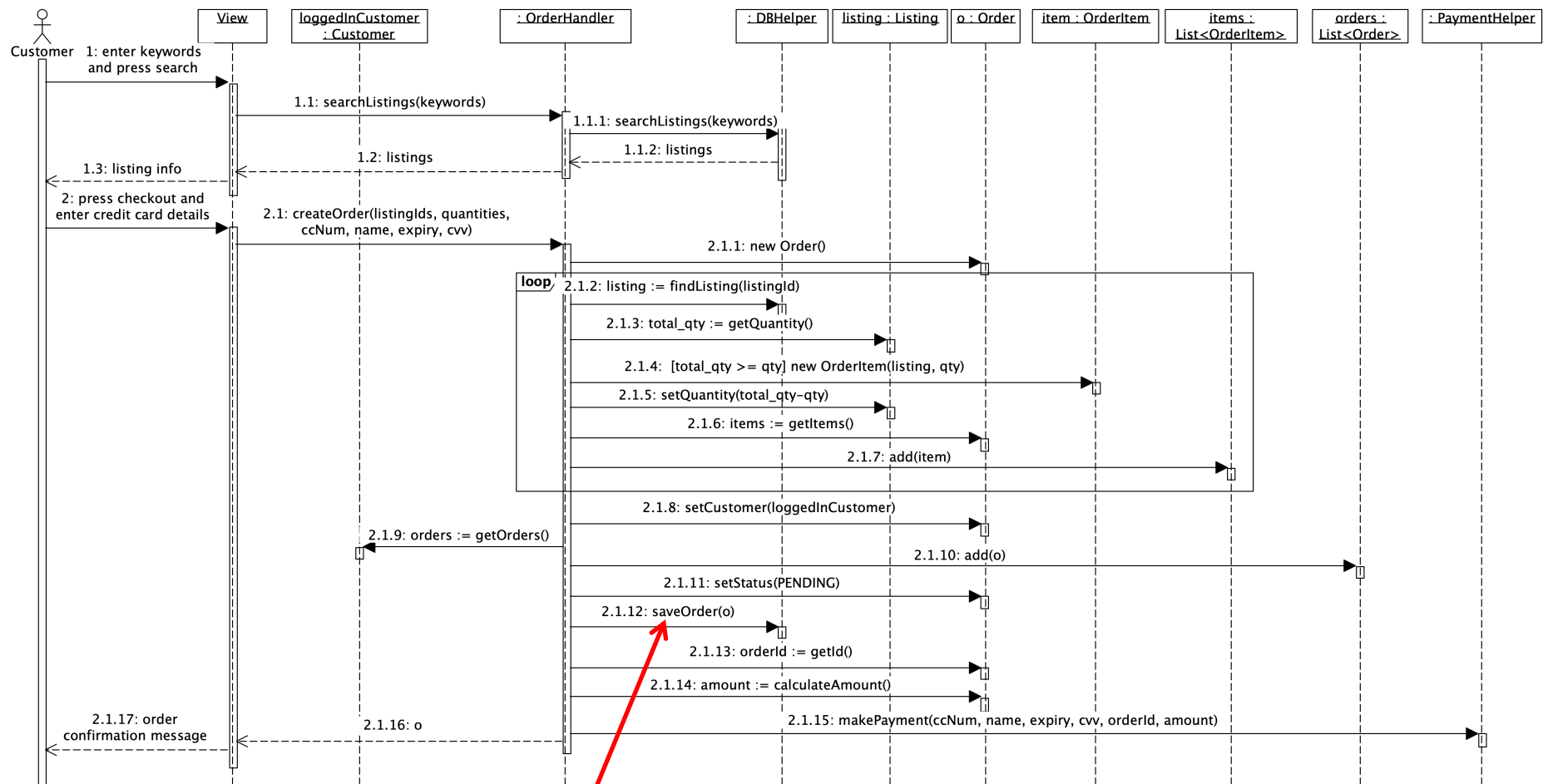
The only time you can write text rather than methods is when the target is the view layer



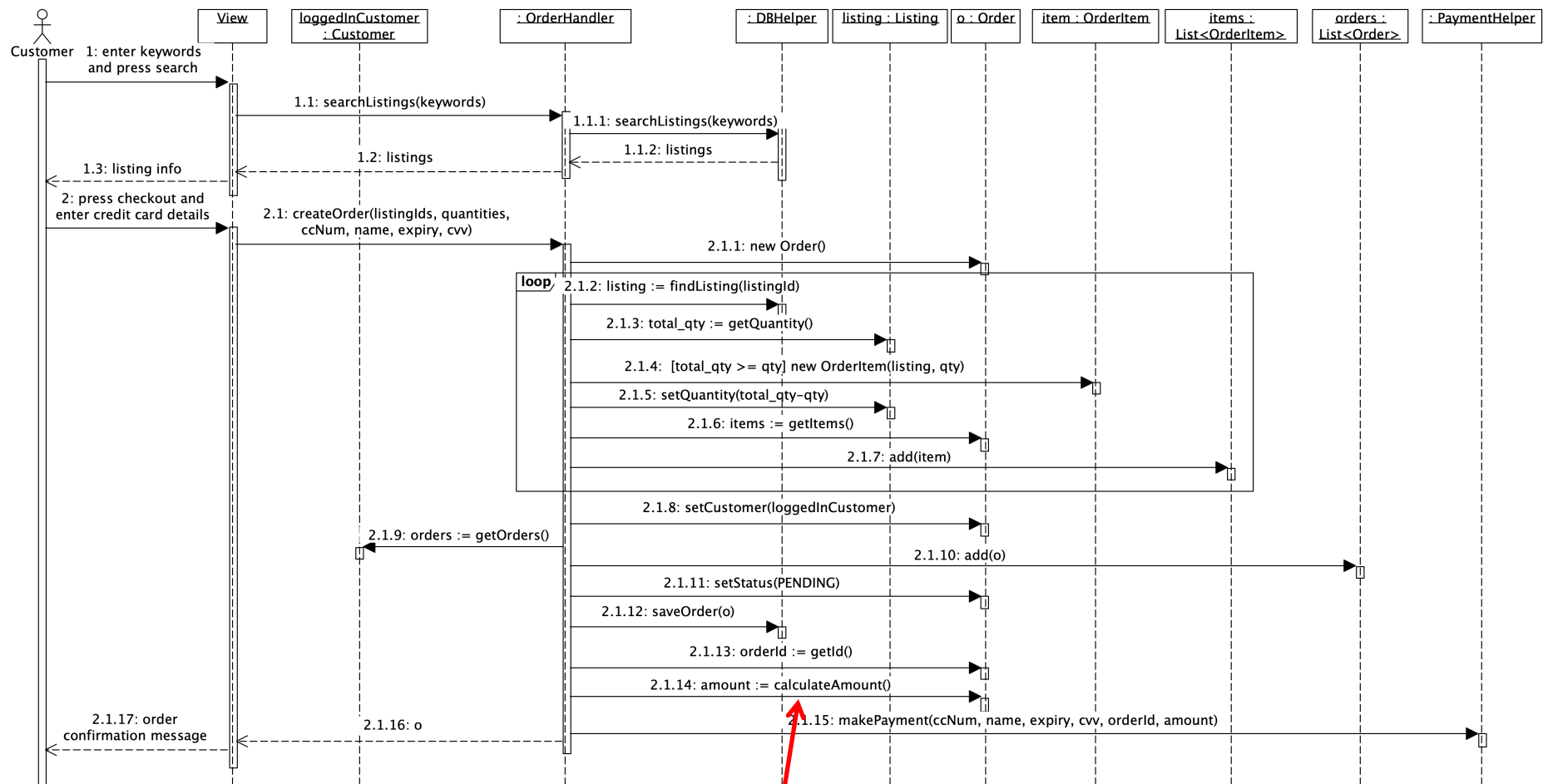
Whatever data that is needed for computation must either be retrieved from the DB or supplied by the user



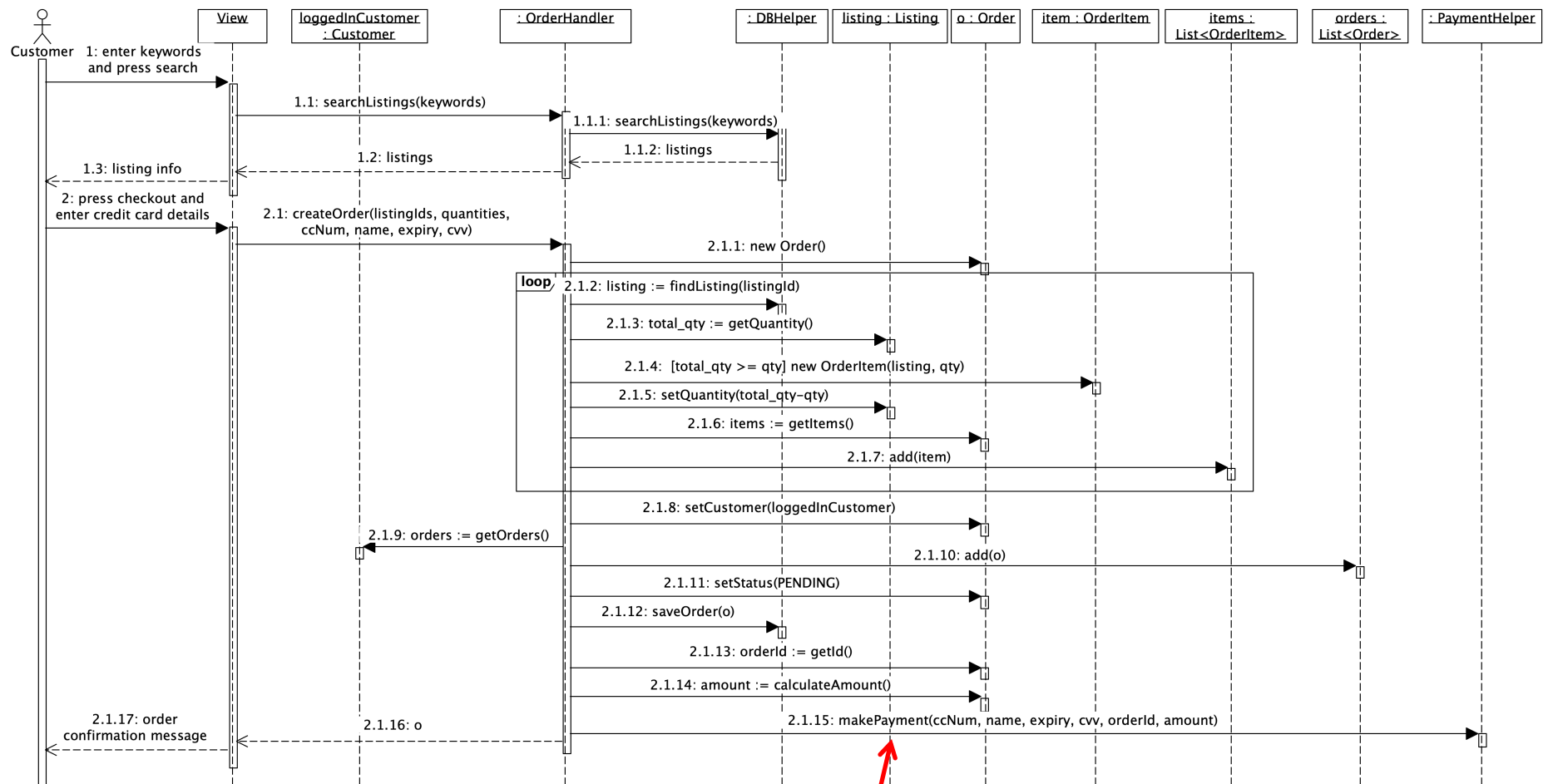
Think Object-Oriented (rather than id unless its supplied by user)



Save the Order object only when it is time to check out



Need to calculate the amount



Asynchronous operation (does not return data back to the caller)



SUMMARY

Covered fundamental design principles (e.g. modularity, encapsulation, etc)

Used JavaEE as an example to demonstrate how enterprise system framework works and how to design systems using its capabilities

Translating use case description into actual design (sequence diagram) with the framework capabilities in mind

WHAT'S NEXT?

Enterprise System Development Architecture

Django Demo